

Software Verification and Validation

Part Time Lecturer
Talimul Bari Shreshtho

Islamic University of Technology

June 19, 2026



Table of Contents

Why Do We Need Verification & Validation?

A Simple Software Error

A research team launched a high-altitude balloon carrying a rocket intended to reach Low Earth Orbit (LEO).

At 95,990 feet, the onboard computer incorrectly concluded that the vehicle was descending and triggered an emergency abort sequence.

- Mission failed
- Significant financial loss
- Loss of stakeholder confidence
- No hardware malfunction occurred

Why Do We Need Verification & Validation?

A Simple Software Error

A research team launched a high-altitude balloon carrying a rocket intended to reach Low Earth Orbit (LEO).

At 95,990 feet, the onboard computer incorrectly concluded that the vehicle was descending and triggered an emergency abort sequence.

- Mission failed
- Significant financial loss
- Loss of stakeholder confidence
- No hardware malfunction occurred

Question: What caused the failure?

The Root Cause

Expected Logic

If two consecutive altitude readings indicate descent, then abort the mission.

Actual Implementation

- A conditional statement used an assignment operator
- Instead of an equality/comparison operator
- The abort condition always evaluated as true

Result

A single coding mistake caused the system to abort a perfectly healthy mission.

- The requirements were reasonable.
- The design considered temporary altitude fluctuations.
- The implementation did not correctly realize the design.
- Testing alone failed to expose the defect before deployment.

Key Question

How can we gain confidence that software behaves as intended before deployment?

Verification

“Are we building the system right?”

Checks whether each development artifact correctly implements the previous one.

Validation

“Are we building the right system?”

Checks whether the system satisfies stakeholder needs and intended use.

What is Verification & Validation?

Definition

Verification and Validation (V&V) is a systems engineering practice that provides confidence that software systems satisfy their requirements and expected operational behavior.

Common V&V techniques include:

- Reviews and inspections
- Static analysis
- Dynamic analysis
- Testing
- Formal methods

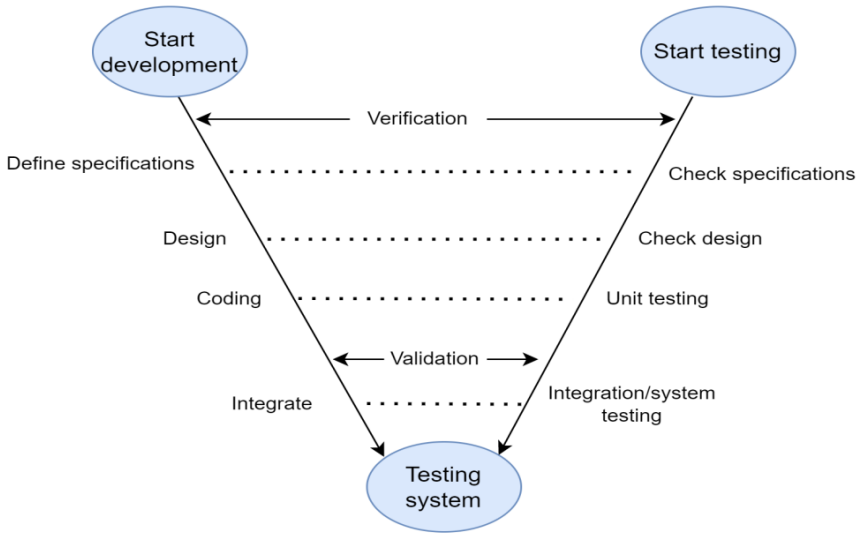
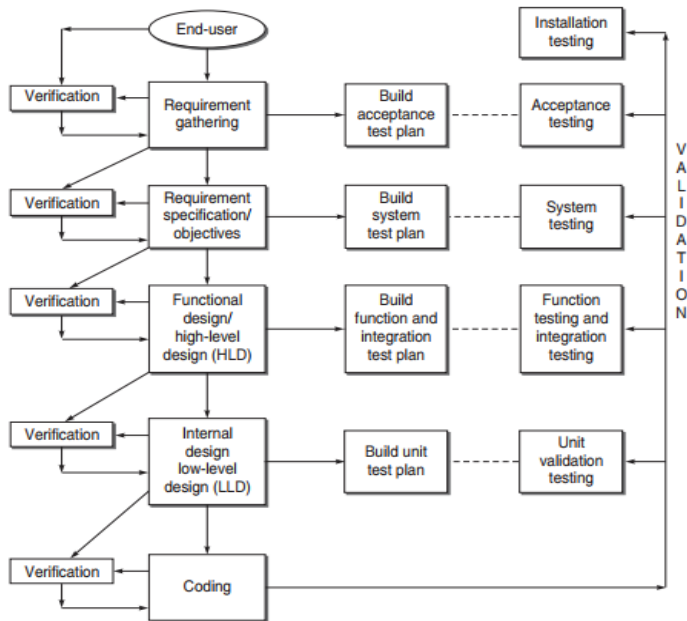


Fig: V-Testing



Verification In testing

Definition

Verification is a **static evaluation process** used to determine whether software artifacts satisfy the requirements and specifications defined for them.

It focuses on:

"Are we building the system right?"

Verification activities are typically performed **without executing the software**.

Common Verification Techniques

- Reviews and inspections
- Walkthroughs
- Static analysis
- Formal verification

Verification of Requirements and Objectives

Requirements Verification

- Verify that user requirements accurately reflect stakeholder needs.
- Review requirements, acceptance criteria, and constraints.
- Identify missing, conflicting, or unclear requirements.
- Prepare the **Acceptance Test Plan**.

Objectives Verification

- Verify that system objectives are derived correctly from the requirements.
- Ensure that every requirement is addressed by one or more objectives.
- Prepare the **System Test Plan**.

A Good SRS Should Be

- Correct • Unambiguous • Consistent • Complete •
- Up-to-date • Traceable

Software Design Verification

- The system architecture and design are documented in the **Software Design Document (SDD)**.
- Verify that the proposed architecture satisfies all system requirements.
- Ensure that system components and their interfaces interact correctly.
- Identify design inconsistencies, missing functionality, and architectural risks.
- Prepare:
 - Functional Test Plan
 - Integration Test Plan

Key Design Elements to Verify

- Data Design
- Architectural Design
- Interface Design

Low-Level Design (LLD)

- The detailed design of each module or component is specified.
- Operational details, algorithms, data structures, and internal logic are documented.
- Each module may have its own detailed requirements and design specifications.
- Verify that every module correctly implements the High-Level Design (HLD).
- Ensure that all functional requirements are addressed at the module level.

Verification Activities

- Check completeness, consistency, correctness, and traceability of the design.
- Review module interactions and internal interfaces.
- Verify error handling and exception processing mechanisms.
- Prepare the **Unit Test Plan** for Unit Testing.

Key Elements to Verify

- Module logic and algorithms
- Data structures and local data design
- Internal interfaces
- Error handling
- Traceability to HLD requirements

Definition

Validation is a **dynamic evaluation process** used to determine whether the developed software satisfies user needs and intended operational requirements.

It focuses on:

"Are we building the right system?"

Validation activities require **execution of the software** and observation of its behavior.

Common Validation Techniques

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

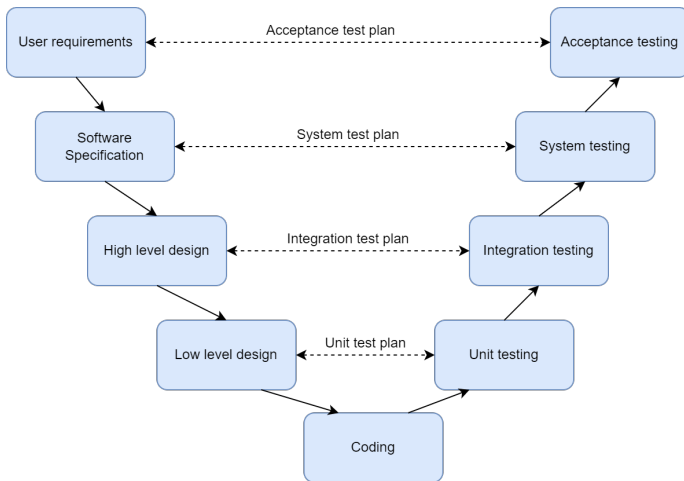
Validation involves executing the tests that were planned during Verification.

- Unit Testing
- Integration Testing
- Functional Testing
- System Testing
- Acceptance Testing

Purpose:

To determine whether the implemented software satisfies user needs and operational requirements.

VV In testing



Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`